HOME ASSIGNMENT · SECURITY ANALYST · UPWIND

MailGuard

A Gmail add-on that scores every email from 0 to 100 and explains, in plain language, why

Looks safe | Suspicious | Likely malicious

github.com/morykenig/mailguard

What the add-on does

Most filters only say “spam” or “not spam”. MailGuard shows the user **why**: every point of the score comes from a specific finding – the link, the phrase or the address in the email itself.

0–100

risk score for every email

3

verdicts: safe · suspicious · malicious

1–2 s

from opening the email to the answer

Flow: from opening an email to the card

Inside the user's Google account

1. Open email
The user opens an email in Gmail

→

2. Collect
Headers, body, attachments, sender history

→

3. Signed request
HMAC + timestamp against forgery and replay

On the backend (Python + FastAPI · Cloud Run)

4. 12 analysers
Each returns findings with points

→

5. Threat intel
VirusTotal · Safe Browsing · domain age

→

6. Score + explain
Summary, advice, MITRE – English or Hebrew

7. Signed response → card in Gmail: the add-on verifies the signature, then shows the banner, gauge, findings and action buttons

What is checked: 12 independent analysers

Sender authentication
SPF · DKIM · DMARC

Sender identity
Look-alike domains, fake ✅ badges

Subject line
Fake “RE:”, alarm tags

Links
Text ≠ target, “oops, real link”

Attachments
Macros, disguised programs

Content, 7 languages
Threats, passwords, payment

Social engineering
Full phishing pattern, CEO fraud

History and memory
First contact, reported before

Images and emojis
Image-only mail, copied logos

Cloud attacks
Device code, OAuth consent

Hidden content and AI
Hidden prompts for AI assistants

Threat intelligence
VirusTotal, Safe Browsing

How the score is calculated

Additive evidence

Each finding adds points; reassuring signs subtract a little. Every point on screen is explained.

A cap per category

Many urgent words alone can't make an email malicious – it takes evidence from several angles.

Floors

Conclusive evidence (a disguised program, a device code) sets a minimum of 70 on its own.

0–29 · Looks safe	30–64 · Suspicious	65–100 · Likely malicious

Confidence (high / medium / low) drops when a check could not run – a missing check is never shown as a clean one.

Live example: “You won cash, click this”

The email

Hey, looks like you won some cash! Please click this!
You can find us here: google.com
Oh oops, its here: danger.com

Suspicious · 37/100

+25 · A “corrected” link replaces a trusted one
First google.com, then “oops” → danger.com

+12 · Promises a prize or cash
“you won some cash”

+8 · Pushes you to click
“Please click this!”

-8 for an authenticated sender · MITRE: T1566.002 · T1036 · T1204.001

What the user sees in Gmail

Card element	What it gives the user
⛔ Hazard banner	The decision at a glance, and the main threat named concretely
[▓▓▓

The add-on protects itself, too

🔏 Signed requests and responses

HMAC + timestamp + one-time nonce: nobody can forge a “safe” verdict or replay an old one

🗑️ The backend stores nothing

No mailbox access. Only URLs, domains and file hashes go to threat intel

🧠 Private memory

Dangerous senders are remembered in the user’s own account, as salted hashes – never in clear text

🤖 No AI in the decision

Hidden text can’t talk the engine out of a warning – prompt injection only raises the score

Every string from the email is escaped before display, and links are shown de-fanged, never clickable.

Why I chose MITRE ATT&CK

🗣️ A shared language with the SOC

“T1566.002” means the same in the add-on, the SIEM and the incident report – no translation needed

📊 Measurement and trends

Reports arrive tagged, so you can count: how much OAuth phishing this month, which technique is rising

🧭 A coverage map

You see which techniques the add-on covers and which it doesn’t – and what to build next

🤖 Attacks on AI, too

MITRE ATLAS covers hidden prompts aimed at AI assistants – a new threat ordinary tagging has no place for

In practice: every rule in the code must be mapped (even “no technique” is an explicit decision), and an automated test fails if a rule ships unmapped.

Every finding is mapped to ATT&CK

Each finding on the card shows its technique id, and SecOps reports carry the ids too.

Technique	Example in the add-on
T1566.002	Spearphishing link – the link text hides the real destination
T1656	Impersonation – look-alike domain, copied logo, CEO fraud
T1528	Token theft – device-code and OAuth phishing that bypass MFA
T1657	Financial theft – an urgent wire-transfer request
AML.T0051	MITRE ATLAS – hidden instructions for AI assistants

Weak points: where the system can fail

Scenario	Why it gets through	What limits the damage
A known contact's account is hacked	"You wrote to them before" lowers the score	Reassurance is capped at 20; strong evidence sets a floor
New wording or an unsupported language	Content detection is pattern-based	Links, authentication and sender are checked separately
Text or a QR code inside an image	No OCR	An image-only email is flagged by itself
A clean link that changes after delivery	Links are never opened (on purpose)	Safe Browsing and VirusTotal, when configured
Encrypted archive, RAR or 7z	The contents can't be inspected	"Encrypted archive" is flagged
The backend is unavailable	Single instance, run locally	"No result is not a safe result"
The shared secret leaks	Stored in Script Properties	Zero-downtime key rotation; OIDC identity would be better

What I would improve with more time

1. Measure before tuning
A labelled phishing corpus, precision and recall, and weights learned from data instead of set by hand

2. Identity instead of a shared secret
A Google-signed OIDC token for each user, so the backend knows who is calling – enabling permissions and an audit trail

3. Close the loop with the SOC
Reports that open a ticket, an analyst verdict, and purging the email from every mailbox

4. See more
OCR and QR decoding, sandboxed link detonation, and a “this is safe” button for false positives

And now — a live demo

A malicious email · a suspicious email · a safe email — in a real Gmail account

github.com/morykenig/mailguard